Trabajo Práctico Nº2

Sintaxis y Semántica de los Lenguajes

Introducción al Lenguaje F#

Tema 2: Benchmark

Grupo 20

Leonardo Benavente

Rodrigo Flor

Pablo Lazaga

Valentín Penna

UTN.BA

UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

Nacimiento de F#

F# Es un lenguaje definido como funcional primero, pero que es de programación multiparadigma, y es fuertemente tipado, diseñado para ser conciso, evitar errores comunes como los nulos y facilitar el trabajo con datos complejos. Este lenguaje fue creado por Don Syme, un investigador de Microsoft Research en Cambridge, Inglaterra.

En diciembre de 2001 Syme, con la idea de llevar el poder del paradigma funcional al ecosistema .NET, comenzó a escribir un compilador para intentar pasar el lenguaje OCaml a .NET, donde tomó la decisión de diseño más determinante del proyecto, que sería que: Los tipos, los valores, las excepciones y los hilos sería, sin ninguna traducción, los mismos de .NET. Poco después el proyecto se renombra F#, en referencia a “Functional” y a “ System F”.

Es decir, en 2002 el lenguaje ya había nacido, pero en principio era un proyecto de investigación sin promesa de comercialización, pero en 2006 empresas financieras adoptaron F# para modelos cuantitativos, y en ese momento fue cuando Microsoft optó por empezar a comercializarlo.

Y fue en 2010 que F# 2.0 se lanzó oficialmente como parte de Visual Studio, asi convirtiendose en un lenguaje soportado formalmente por Microsoft, al mismo nivel que C#. Ese mismo año Microsoft liberó el código fuente del compilador y las librerías bajo licencia de código abierto, y en 2014 comenzó a aceptar contribuciones externas de la comunidad.

Por último en 2016 se lanza .NET Core, donde F# Se convierte en un lenguaje totalmente multiplataforma y ejecutable de forma nativa en Linux, MacOS y Windows. Este lenguaje en parte también es importante por su influencia directa en la evolución de C# y otros lenguajes de la industria.

Características principales de F#

Paradigmas

F# es un lenguaje multiparadigma, el cual puede trabajar tanto con el paradigma imperativo, como con el paradigma orientado a objetos, pero sobretodo se caracteriza por el paradigma funcional, esto conlleva que F# tome varios de los rasgos más importantes de este último.

1) Todo es una expresión que puede ser evaluada y devuelve una respuesta.
2) Las variables son inmutables por defecto.
3) Se favorece la recursión en sus funciones
4) Tiene una inferencia de tipos Hindley-Milner, por la cual el sistema deduce el tipo de los datos automáticamente sin necesidad de que el usuario tenga que explicitarlos.

Como consecuencia de esta naturaleza, F# se consolida como un lenguaje robusto,seguro y declarativo.

Plataforma

Este lenguajes es soportado por la plataforma .NET de forma nativa, es decir que su compilador traduce el código a CIL (Common Intermediate Language), por lo cual tiene una gran compatibilidad con C# a la hora de realizar proyectos, y puede acceder a prácticamente todo el catálogo de bibliotecas de este.

Open Source

Su característica de ser open source permite a cualquier desarrollador modificar y mejorar los aspectos que desee del lenguaje, además aun hasta el dia de hoy sigue siendo mantenida por el equipo de Microsoft y la F# Software Foundation, quienes le dan soporte en Windows,Linux y MacOs .

Comparación entre F# y C#

Similitudes:

- Ambos lenguajes forman parte del ecosistema .NET y son soportados por la plataforma de Microsoft.
- Se ejecutan sobre el mismo entorno de ejecución, el **Common Language Runtime (CLR)**.
- Ambos poseen un sistema de tipos estático y fuerte.
- Los dos son lenguajes multiparadigma. Aunque cada uno prioriza uno diferente.

Diferencias:

- C# está orientado principalmente a la programación orientada a objetos e imperativa, mientras que F# prioriza el paradigma funcional.
- En F#, los valores son **inmutables**, reduciendo efectos secundarios y facilitando la concurrencia. En C#, los datos son mutables.
- F# posee una inferencia de tipos mucho más completa que C#, permitiendo escribir menos código sin perder seguridad de tipos.
- F# no permite la existencia de registros nulos en la ejecución, en caso de haber un registro nulo se utiliza el tipo seguro Option<'T>, de esta forma se evitan excepciones que en C# pueden ocurrir por accesos inválidos.
- Su sintaxis varía por su distinta especialización de paradigmas.

Benchmark F# vs C#

BenchmarkDotNet

Una forma sencilla de realizar un benchmark consiste en medir el tiempo utilizando la clase Stopwatch. Este método podría introducir errores de medición debido a la compilación en tiempo de ejecución, esta técnica permite traducir el código IL a instrucciones máquina durante la ejecución del programa.
Por este motivo se utilizó BenchmarkDotNet, una librería para el ecosistema .NET diseñada para medir el rendimiento de métodos. Para lograrlo, ejecuta múltiples iteraciones del mismo método y recopila métricas del proceso como el tiempo que tarda en ejecutarse, el consumo de memoria y otras estadísticas.

Metodología usada en el Benchmark

Búsqueda binaria: Se generó un array de 100.000 enteros ordenados y luego se pidió buscar el mismo elemento en ambas implementaciones, en los dos casos se utilizó el estilo imperativo para observar que tanto se acerca su rendimiento

Quicksort: Para obtener una comparación justa entre los lenguajes, se generó un array de 1.000.000 de enteros aleatorios utilizando la misma seed tanto en C# como en F#. De esta manera, ambas implementaciones procesan exactamente el mismo conjunto de datos, evitando el sesgo que podría surgir al ordenar arreglos con valores diferentes.

Los benchmarks se ejecutaron múltiples veces de forma automática. A partir de dichas ejecuciones, BenchmarkDotNet calculó estadísticas como el tiempo promedio (*Mean*), la desviación estándar (*StdDev*), la mediana (*Median*), la cantidad de memoria asignada (*Allocated*) y la actividad del garbage collector (*Gen0*, *Gen1* y *Gen2*).

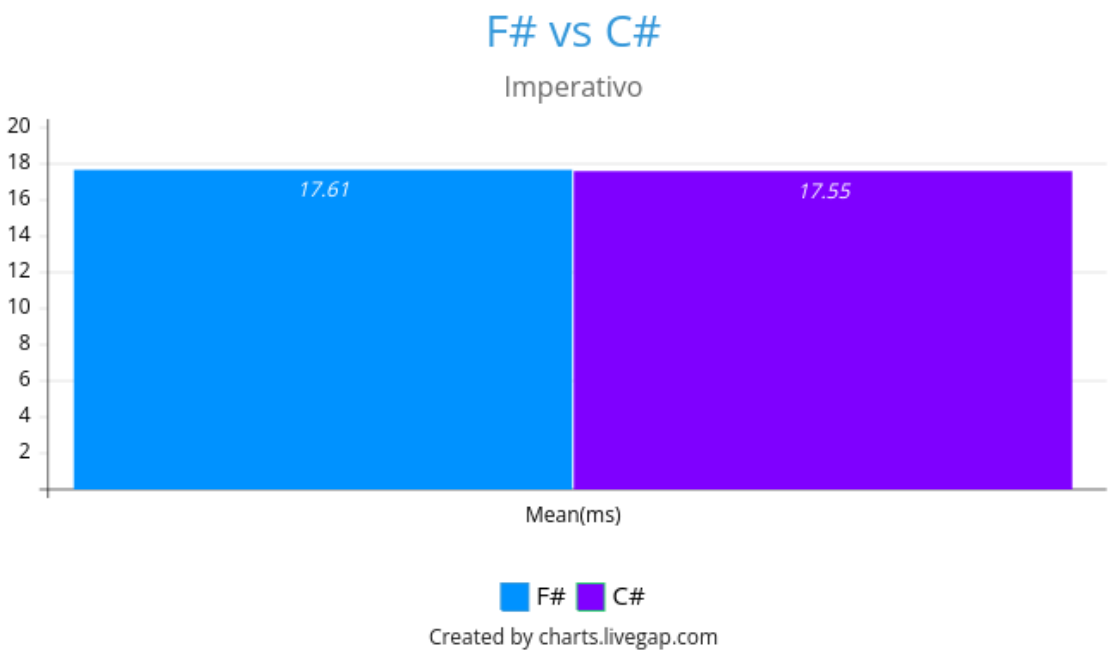
Resultados Esperados

Búsqueda Binaria: Se espera que ambas implementaciones tengan un rendimiento similar,dado que en las dos se usó el paradigma imperativo y puesto que F# y C# comparten el mismo framework (.NET) y runtime (CLR) deberían dar tiempos muy cercanos

Quicksort: Se espera que la implementación en F# tenga un mayor consumo de memoria y un mayor tiempo de ejecución que la implementación en C#.
Esto se debe a que el algoritmo funcional de F# crea nuevos arreglos en cada partición del QuickSort, mientras que la versión imperativa de C# modifica el arreglo original, evitando asignaciones adicionales de memoria.
No obstante, si comparamos el algoritmo en ambos lenguajes bajo el mismo paradigma, la diferencia debería ser nula, ya que tanto C# como F# se ejecutan sobre la plataforma .NET y utilizan el mismo entorno de ejecución (CLR).

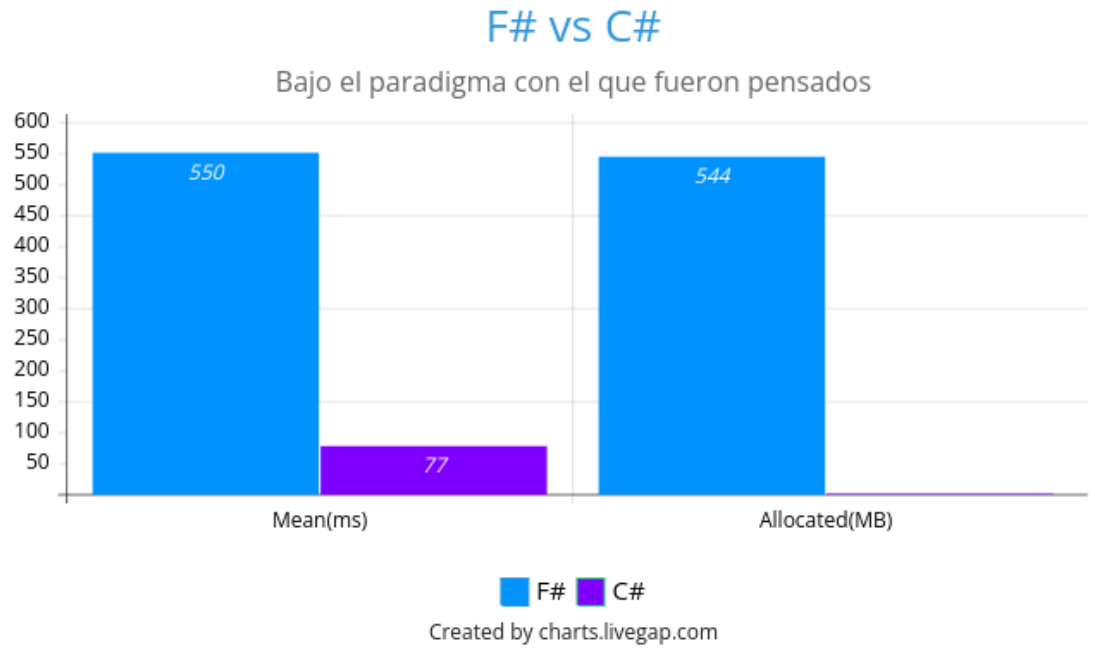
Resultados Obtenidos

Búsqueda Binaria:

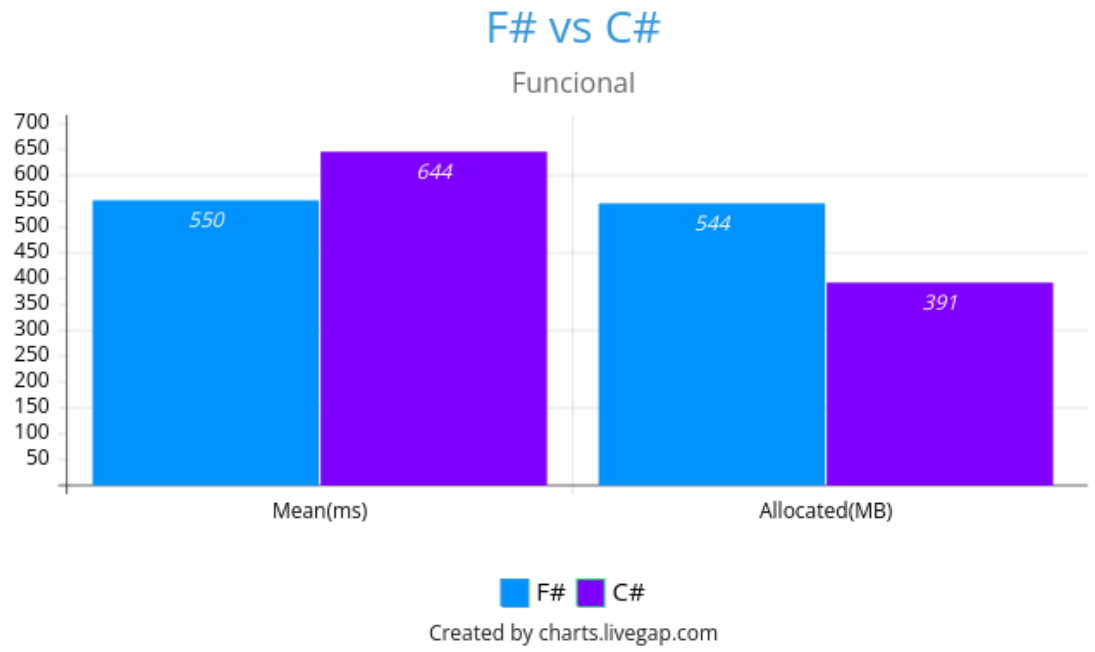


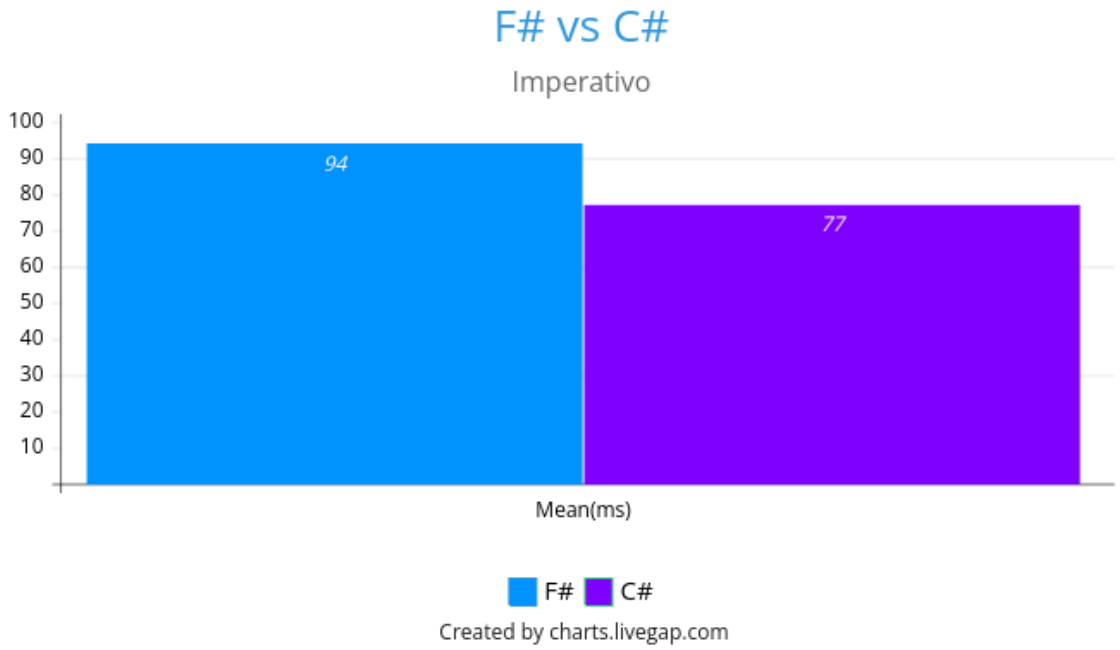
Como se puede ver, los tiempos de ejecución de este algoritmo son prácticamente idénticos en F# y C#, esto concuerda con lo planteado anteriormente y se debe a su semejanza en el entorno de ejecución

Quicksort:



Se puede observar que en el gráfico de la izquierda(Mean) que el algoritmo funcional ejecutado en F# tuvo un tiempo promedio de ejecución de 550.12ms mientras que el QuickSort imperativo de C# tardó en ejecutarse en un promedio de 77.49ms. Es decir, C# fue alrededor de 7.1 veces más rápido que F#.
Mientras que el gráfico de la derecha(Allocated) se observa que F# en el heap reservo 544.4 MB mientras que en C# fue nulo.





Mientras que si comparamos el algoritmo en ambos lenguajes bajo el mismo paradigma de programación observamos que tanto la memoria asignada en el heap como el tiempo promedio de ejecución es significativamente similar.

Conclusiones Finales

En este trabajo se comparó el rendimiento del algoritmo Quicksort implementados en F# y C#, aprovechando el paradigma funcional del primero y lo imperativo del segundo.

Los resultados muestran que la implementación imperativa en C# fue, en promedio, **siete veces** más rápida que la solución funcional de F#. Resultado que era esperable, ya que en la programación funcional los datos son inmutables, creando nuevas estructuras durante la ejecución del algoritmo, mientras que, la implementación imperativa modifica el arreglo original.

Cabe destacar que, al utilizar el mismo paradigma en ambos lenguajes, tanto el rendimiento como el consumo de recursos resultan muy similares. Esto se debe a que C# y F# comparten la plataforma .NET.

Sin embargo, el rendimiento no es el único aspecto que debemos tener en cuenta. La implementación funcional permitió expresar el algoritmo de una forma más natural, obteniendo un código fácil de comprender y, en consecuencia, más sencillo de mantener.

En conclusión, mientras que la solución imperativa demuestra un mayor rendimiento y un mejor aprovechamiento de los recursos, la solución funcional ofrece un código más declarativo, legible y fácil de mantener.